a. GPT-5.6 and Gemini 3

b. Each from their respective websites

c. generate C++ code using objects, not functions, to implement
the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-

grids, or blocks.

- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject

to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.

- Published puzzles are usually constructed to have a unique solution. This

requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.

- The Sudoku problem, then, is this: Given a partially completed Sudoku

puzzle, find the (or at least a) solution; if no solution exists, prove that.

- It is possible to solve a Sudoku puzzle entirely through logic. For example, in

the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.

- That is not the approach we’re going to use. Instead, we’re going to

implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)

- Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells

are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

- This approach, by the way, is called depth-first search with backtracking. For

each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.

- You are given five files (puzzle1.txt through puzzle5.txt), each containing a

Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.

- Write a recursive object-oriented program to find the solution to the five

puzzle files.

- The output for each solution, should include:

o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.

- The solution may not be unique; if there is more than one solution to the

puzzle, your program should print them all out.

d.

GEMINI CODE:

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
```

```
class SudokuBoard {
private:
    std::vector<std::vector<char> > grid;

public:
    SudokuBoard() : grid(9, std::vector<char>(9, '_')) {}

    // Loads the puzzle from a file, reading 81 valid characters ('1'-'9' or '_')
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }
        char ch;
        int count = 0;
        while (file >> ch && count < 81) {
            if (ch == '_' || (ch >= '1' && ch <= '9')) {
                int r = count / 9;
                int c = count % 9;
                grid[r][c] = ch;
                count++;
            }
        }
        return count == 81;
    }

    // Prints the board formatted neatly to the console
    void print() const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                std::cout << grid[r][c] << (c == 8 ? "" : " ");
            }
            std::cout << "\n";
        }
    }

    // Finds the next unassigned (empty) cell marked with '_'
    bool findUnassignedLocation(int& row, int& col) const {
        for (row = 0; row < 9; ++row) {
            for (col = 0; col < 9; ++col) {
                if (grid[row][col] == '_') {
                    return true;
                }
```

```
            }
        }
        return false;
    }

    // Validates if placing a number at a given cell violates Sudoku rules
    bool isValidPlacement(int row, int col, char num) const {
        // Check row constraints
        for (int c = 0; c < 9; ++c) {
            if (grid[row][c] == num) return false;
        }

        // Check column constraints
        for (int r = 0; r < 9; ++r) {
            if (grid[r][col] == num) return false;
        }

        // Check 3x3 block constraints
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (grid[startRow + r][startCol + c] == num) return false;
            }
        }

        return true;
    }

    char getCell(int r, int c) const {
        return grid[r][c];
    }

    void setCell(int r, int c, char val) {
        grid[r][c] = val;
    }
};

class SudokuSolver {
private:
    SudokuBoard initialBoard;
    std::vector<SudokuBoard> solutions;

    // Recursive backtracking method to discover all valid solutions
```

```
    void solveRecursive(SudokuBoard& currentBoard) {
        int row, col;
        // Base case: if no unassigned cells remain, a solution is found
        if (!currentBoard.findUnassignedLocation(row, col)) {
            solutions.push_back(currentBoard);
            return;
        }

        // Try digits '1' through '9'
        for (char digit = '1'; digit <= '9'; ++digit) {
            if (currentBoard.isValidPlacement(row, col, digit)) {
                currentBoard.setCell(row, col, digit);

                // Recurse with the candidate placed
                solveRecursive(currentBoard);

                // Backtrack: reset cell to empty and try the next candidate
                currentBoard.setCell(row, col, '_');
            }
        }
    }

public:
    bool loadPuzzle(const std::string& filename) {
        solutions.clear();
        return initialBoard.loadFromFile(filename);
    }

    void solve() {
        solutions.clear();
        SudokuBoard tempBoard = initialBoard;
        solveRecursive(tempBoard);
    }

    // Displays file name, initial puzzle, and all discovered solutions (or none)
    void displayResults(const std::string& filename) const {
        std::cout << "========================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "----------------------------------------\n";
        std::cout << "Initial Puzzle:\n";
        initialBoard.print();
        std::cout << "----------------------------------------\n";

        if (solutions.empty()) {
```

```
            std::cout << "No solution found.\n";
        } else {
            std::cout << "Found " << solutions.size() << " solution(s):\n";
            for (size_t i = 0; i < solutions.size(); ++i) {
                if (solutions.size() > 1) {
                    std::cout << "Solution " << (i + 1) << ":\n";
                }
                solutions[i].print();
                if (i + 1 < solutions.size()) {
                    std::cout << "----------------------------------------\n";
                }
            }
        }
        std::cout << "========================================\n\n";
    }
};

int main() {
    std::string puzzleFiles[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt",
"puzzle5.txt"};

    for (const auto& filename : puzzleFiles) {
        SudokuSolver solver;
        if (solver.loadPuzzle(filename)) {
            solver.solve();
            solver.displayResults(filename);
        } else {
            std::cout << "Error: Could not open or read " << filename << ".\n\n";
        }
    }

    return 0;
}
```

GPT CODE:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cctype>
```

```
using namespace std;

class Sudoku {
private:
    int grid[9][9];
    long long solutionsFound;

    // Check whether a digit can be placed in a cell.
    bool isSafe(int row, int col, int digit) const {
        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == digit) {
                return false;
            }

            if (grid[i][col] == digit) {
                return false;
            }
        }

        // Find the top-left cell of the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // Check all cells in that 3x3 block.
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (grid[r][c] == digit) {
                    return false;
                }
            }
        }

        return true;
    }

    // Find the next empty cell.
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {
                if (grid[row][col] == 0) {
                    return true;
                }
```

```
            }
        }

        return false;
    }

    // Recursive backtracking: find and print ALL solutions.
    void solveAll() {
        int row, col;

        // Base case: no empty cells remain.
        if (!findEmptyCell(row, col)) {
            solutionsFound++;

            cout << "\nSolution #" << solutionsFound << ":\n";
            printBoard();

            return;
        }

        // Try each possible digit.
        for (int digit = 1; digit <= 9; digit++) {
            if (isSafe(row, col, digit)) {
                // Make a choice.
                grid[row][col] = digit;

                // Recursively solve the remaining puzzle.
                solveAll();

                // Backtrack: undo the choice and try another digit.
                grid[row][col] = 0;
            }
        }
    }

public:
    // Constructor: initialize the board.
    Sudoku() {
        solutionsFound = 0;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                grid[row][col] = 0;
            }
```

```
    }
}

// Read the puzzle file.
// Underscores represent empty cells.
bool loadFromFile(const string& filename) {
    ifstream input(filename);

    if (!input.is_open()) {
        return false;
    }

    int cellCount = 0;
    char ch;

    while (input.get(ch)) {
        // Ignore spaces, tabs, and newlines.
        if (isspace(static_cast<unsigned char>(ch))) {
            continue;
        }

        // Reject files containing more than 81 cells.
        if (cellCount >= 81) {
            input.close();
            return false;
        }

        // Convert the next character into a cell value.
        if (ch == '_') {
            grid[cellCount / 9][cellCount % 9] = 0;
        } else if (ch >= '1' && ch <= '9') {
            grid[cellCount / 9][cellCount % 9] = ch - '0';
        } else {
            input.close();
            return false;
        }

        cellCount++;
    }

    input.close();

    // A Sudoku puzzle must contain exactly 81 cells.
    return cellCount == 81;
```

```
}

// Print the current Sudoku board.
void printBoard() const {
    cout << "+-------+-------+-------+\n";

    for (int row = 0; row < 9; row++) {
        cout << "| ";

        for (int col = 0; col < 9; col++) {
            if (grid[row][col] == 0) {
                cout << "_ ";
            } else {
                cout << grid[row][col] << " ";
            }

            if ((col + 1) % 3 == 0) {
                cout << "| ";
            }
        }

        cout << "\n";

        if ((row + 1) % 3 == 0) {
            cout << "+-------+-------+-------+\n";
        }
    }
}

// Check whether the original clues conflict.
bool hasValidClues() const {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            int digit = grid[row][col];

            // Empty cells cannot create clue conflicts.
            if (digit == 0) {
                continue;
            }

            // Check the rest of this row and column.
            for (int i = 0; i < 9; i++) {
                if (i != col && grid[row][i] == digit) {
                    return false;
```

```
            }

            if (i != row && grid[i][col] == digit) {
                return false;
            }
        }

        // Check the rest of this 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if ((r != row || c != col) &&
                    grid[r][c] == digit) {
                    return false;
                }
            }
        }
      }
    }

    return true;
  }

  // Solve the puzzle and print all solutions.
  void solvePuzzle() {
    solutionsFound = 0;

    solveAll();

    if (solutionsFound == 0) {
      cout << "\nNo solution found.\n";
    } else {
      cout << "\nTotal solutions found: "
           << solutionsFound << "\n";
    }
  }
};

int main() {
  // Process all five puzzle files.
  for (int i = 1; i <= 5; i++) {
    string filename = "puzzle" + to_string(i) + ".txt";
```

```
        cout << "\n==================================\n";
        cout << "File: " << filename << "\n";
        cout << "==================================\n";

        // Create a Sudoku object for this puzzle.
        Sudoku puzzle;

        // Load the puzzle from its file.
        if (!puzzle.loadFromFile(filename)) {
            cout << "Unable to read " << filename
                 << ". Check that the file exists and "
                 << "contains exactly 81 valid cells.\n";
            continue;
        }

        // Print the original puzzle.
        cout << "\nOriginal puzzle:\n";
        puzzle.printBoard();

        // Reject puzzles with conflicting starting clues.
        if (!puzzle.hasValidClues()) {
            cout << "\nNo solution found.\n";
            continue;
        }

        // Find and print every solution.
        puzzle.solvePuzzle();
    }

    return 0;
}
```

e.

Correctness: Although Gemini takes the sample inputs provided and correctly outputs GPT also works but it has some additional benefits such as rejecting any character other than 1-9 or _, rejects files more than 81 cells, and checks whether the starting clues already conflict

Execution Time: Both use the same brute force backtracking with the same first empty cell scan with GPT being slightly faster on puzzles with conflicting clues because it stops before searching

Space Complexity: GPT prints each solution as it finds it so it uses no extra memory while Gemini stores a full board copy for every solution before printing so its memory grows with the number of solutions

Maintainability: Gemini has clearer object design while GPT has better comments and a clearer boxed output but one class does loading solving and printing and the solver prints from inside the recursion

f. I'd select GPT as the base because it can correctly process the given inputs and inputs outside the given inputs all while using less memory

g.
Stopped the search early when a blank cell has no valid candidates and kept the pre-search clue check so invalid puzzles are rejected immediately

Adopted some of Geminis strategy of splitting the single Sudoku class into a board class with grid, load, print, clue check, and isSafe and a solver class

Made an error message saying why a file is rejected

Added more comments for clarity